

Phase 4 — PostgreSQL & Django Integration

Revision Notes

This phase covered PostgreSQL installation and setup, SQL fundamentals, relationships, constraints, indexes, transactions, and connecting Django to PostgreSQL.

1. Install PostgreSQL

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl status postgresql
sudo systemctl start postgresql
sudo systemctl enable postgresql
psql --version
```

2. Open PostgreSQL

```
sudo -u postgres psql
\l
\du
\dn
\q
```

3. Create User and Database

```
CREATE USER myproject_user WITH PASSWORD 'your_password';
CREATE DATABASE myproject_db;
GRANT ALL PRIVILEGES ON DATABASE myproject_db TO myproject_user;
```

Connect to the project database:

```
psql -U myproject_user -d myproject_db -h localhost
```

4. Schema Permissions

```
GRANT USAGE, CREATE ON SCHEMA public TO myproject_user;
```

If permission problems occur, check the current PostgreSQL user with `SELECT current_user;` and perform grants from an appropriate database owner/admin connection.

5. Basic SQL

```
CREATE TABLE test_product (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    price INTEGER
);

INSERT INTO test_product (name, price)
VALUES ('Laptop', 800);

SELECT * FROM test_product;

UPDATE test_product
SET price = 900
WHERE id = 1;

DELETE FROM test_product
WHERE id = 1;
```

Also practiced WHERE, AND, OR, IN, ORDER BY, LIMIT, COUNT, SUM, AVG, MIN, MAX, GROUP BY and HAVING.

6. Keys and Relationships

A PRIMARY KEY uniquely identifies a row. A FOREIGN KEY connects rows between tables and protects referential integrity.

```
CREATE TABLE test_customer (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100)  
);  
  
CREATE TABLE test_order (  
    id SERIAL PRIMARY KEY,  
    customer_id INTEGER REFERENCES test_customer(id),  
    product_id INTEGER REFERENCES test_product(id)  
);
```

JOINS covered: INNER JOIN for matching records and LEFT JOIN for all rows from the left table plus matching rows where available.

7. Constraints, Indexes and Transactions

Important constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE and CHECK. Indexes can speed up suitable lookups. while adding storage and write-maintenance cost.

```
CREATE INDEX product_name_index  
ON test_product(name);
```

Transactions:

```
BEGIN;  
UPDATE test_product SET price = 999 WHERE id = 1;  
ROLLBACK;
```

Use COMMIT instead of ROLLBACK when the transaction should be saved.

8. Install the Django PostgreSQL Driver

```
source .venv/bin/activate  
pip install psycopg2  
pip show psycopg2
```

9. Configure Django

In settings.py. replace the SQLite DATABASES configuration with:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'myproject_db',  
        'USER': 'myproject_user',  
        'PASSWORD': 'YOUR_POSTGRES_PASSWORD',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

For real projects, keep passwords in environment variables rather than committing them to source control.

10. Run Django Checks and Migrations

```
python manage.py check  
python manage.py makemigrations  
python manage.py migrate
```

migrate creates Django's required database tables and your application's tables inside PostgreSQL.

11. Verify Django Uses PostgreSQL

```
python manage.py shell  
  
from django.db import connection  
connection.vendor
```

```
connection.settings_dict["NAME"]
connection.settings_dict["HOST"]
connection.settings_dict["PORT"]
```

Expected: vendor = postgresql, database = myproject_db, host = localhost, port = 5432.

12. Inspect SQL Generated by Django ORM

```
from myapp.models import Product

queryset = Product.objects.all()
print(queryset.query)
```

Conceptually: Django ORM → SQL → psycopg → PostgreSQL → result → Django objects.

13. Verify Data End-to-End

```
product = Product.objects.create(
    name="PostgreSQL Laptop",
    price=1000
)

Product.objects.all()
```

Exit the Django shell, connect with psql, run \dt, find the application's Product table, and use SELECT * FROM ; to verify the same row directly in PostgreSQL.

14. Essential PostgreSQL Commands

```
\l                -- list databases
\c database_name  -- connect to a database
\dt               -- list tables
\d table_name     -- describe a table
\dn               -- list schemas
\du               -- list roles/users
\q               -- quit
SELECT current_user;
SELECT current_database();
SELECT version();
```

15. Linux Service Commands

```
sudo systemctl status postgresql
sudo systemctl start postgresql
sudo systemctl stop postgresql
sudo systemctl restart postgresql
sudo systemctl enable postgresql
```

16. Key Takeaways

- PostgreSQL is the database that stores and processes the application's data.
- Django ORM provides a Python abstraction over database queries.
- Migrations translate Django model changes into database schema changes.
- Primary keys identify rows and foreign keys create relationships.
- Constraints protect data integrity.
- Indexes can improve suitable read queries.
- Transactions provide COMMIT and ROLLBACK control.
- Django connects to PostgreSQL through the psycopg driver.

Phase Status

COMPLETED — PostgreSQL fundamentals and Django integration.

Next phase: Authentication & Authorization.